

mecaniQA – Aracaju

Autor: João Victor Carvalho de Oliveira
Kaio Santo de Oliveira
Marco Antonio Almeida Silva
Nivton Amaral Alves
Robert de Araújo Costa

Agenda

O objetivo desta apresentação é apresentar a Web API desenvolvida para a mecaniQA, destacando sua modelagem, funcionalidades e principais conceitos utilizados.

1. Contexto e arquitetura

Estrutura geral da API e fluxo das requisições

2. Modelagem das classes

Entidades, enumeração, controllers e repositories.

3. CRUD e rotas REST

Operações de cadastro, consulta, atualização e exclusão.

4. Singleton e persistência

Armazenamento em memória e gerenciamento das instâncias

5. Validação e tratamento de erros

Validação dos dados e utilização dos status HTTP

Contexto e Arquitetura da API



Objetivo do projeto

Desenvolver uma Web API REST para gerenciamento de Peças e Serviços da mecaniQA

Tecnologias

Java 21
Spring Boot
Gradle

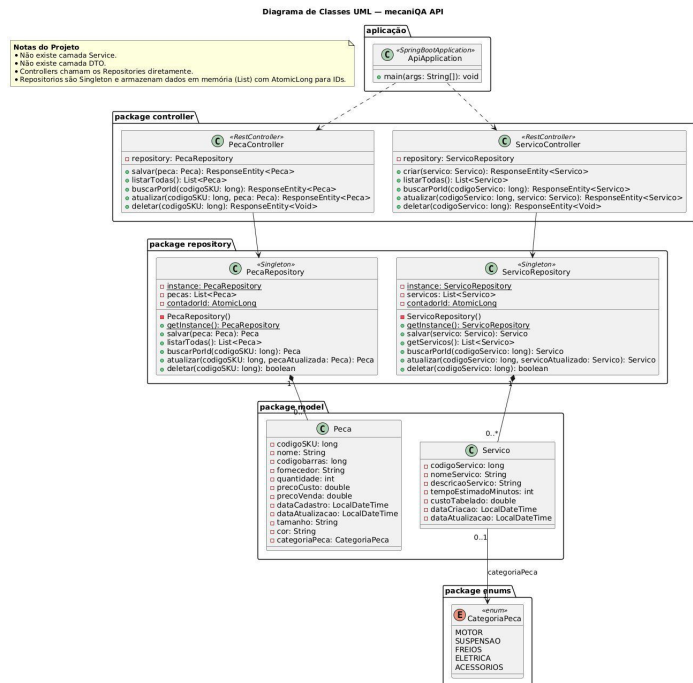
Arquitetura

Cliente / Postman
↓
Controller
↓
Repository
↓
Dados em memória

```
16 @RestController
17 @RequestMapping("/api/pecas")
18 public class PecaController {
19
20     private final PecaRepository repository = PecaRepository.getInstance();
21
22     // ===== CREATE =====
23     @PostMapping
24     public ResponseEntity<Peca> salvar(@Valid @RequestBody Peca peca) {
25         Peca pecaSalva = repository.salvar(peca);
26         return ResponseEntity.status(HttpStatus.CREATED).body(pecaSalva);
27     }
```



Modelagem de classes



POO no projeto

- Classes e atributos
- Encapsulamento com private
- Construtor
- Getters e Setters
- Enum CategoriaPeca

Conceitos de POO

Classes e atributos

Peca e **Servico** representam as principais entidades da API, contendo seus respectivos atributos.

Encapsulamento

Atributos são acessados por meio de Getters e Setters.

Construtores

Utilizados para inicializar os objetos.

Enum

CategoriaPeca define as categorias possíveis para as peças.

Membros static

Utilizados no Singleton para controlar a instância compartilhada do Repository

Peca.java

```
14 public class Peca {
15
16     // ===== ATRIBUTOS =====
17     private long codigoSKU;
18
19     @NotBlank(message = "O nome da peça é obrigatório")
20     private String nome;
21
22
23     // ===== CONSTRUTOR =====
24
25     public Peca() {
26     }
27
28
29     // ===== GETTERS =====
30
31     public long getCodigoSKU() {
32         return codigoSKU;
33     }
34
35
36     // ===== SETTERS =====
37
38     public void setCodigoSKU(long novoCodigo) {
39         codigoSKU = novoCodigo;
40     }
41 }
```

CRUD e Rotas REST



Controllers: responsáveis por receber as requisições HTTP e disponibilizar as rotas e endpoints da API

Peças

POST /api/pecas
Criar peça

GET /api/pecas
Listar peças

GET /api/pecas/{id}
Buscar peça

PUT /api/pecas/{id}
Atualizar peça

DELETE /api/pecas/{id}
Excluir peça

Serviços

POST /api/servicos
Criar serviço

GET /api/servicos
Listar serviços

GET /api/servicos/{id}
Buscar serviço

PUT /api/servicos/{id}
Atualizar serviço

DELETE /api/servicos/{id}
Excluir serviço

Status HTTP

201 Created
Cadastro realizado

200 OK
Consulta / atualização

204 No Content
Exclusão realizada

404 Not Found
Recurso não encontrado

400 Bad Request
Dados inválidos

POST → criação
GET → consulta
PUT → atualização
DELETE → exclusão

Singleton, Validação e Tratamento de Erros

Singleton

Como funciona: o construtor privado impede novas instâncias, enquanto o método `getInstance()` fornece acesso à instância única compartilhada.

- Construtor privado
- Instância única
- Método `getInstance()`
- Dados mantidos em memória

Validação

- `@Valid`
- `@NotBlank`
- `@NotNull`
- `@Positive`
- `@PositiveOrZero`

Tratamento de erros

- 400 Bad Request
→ Dados inválidos
- 404 Not Found
→ Recurso não encontrado

PecaRepository.java

```
16 @RestController
17 @RequestMapping("/api/pecas")
18 public class PecaController {
19
20     private final PecaRepository repository = PecaRepository.getInstance();
21
22     // ===== CREATE =====
23     @PostMapping
24     public ResponseEntity<Peca> salvar(@Valid @RequestBody Peca peca) {
25         Peca pecaSalva = repository.salvar(peca);
26         return ResponseEntity.status(HttpStatus.CREATED).body(pecaSalva);
27     }
```

Considerações finais



Principais resultados

- Implementação do CRUD de Peças e Serviços
- Criação de rotas REST
- Modelagem das entidades utilizando classes e enumeração
- Implementação do Padrão Singleton
- Persistência dos dados em memória
- Validação das requisições
- Tratamento de exceções
- Utilização adequada dos status HTTP

O projeto estabelece a base da API da mecaniQA para as próximas etapas do desenvolvimento.

Referências



MECANIQA AUTOMOTIVE TECH.

OAT 1 — Introdução a Diagrama de
Classes. Programa de Trainee
2026.2.

GitHub.

mecaniQA-api-aracaju. Repositório
do projeto da equipe.

[github.com/JottavDEV/mecani
QA-api-aracaju](https://github.com/JottavDEV/mecaniQA-api-aracaju)